

面向 LLM / Agent Systems 科研的深度学习零基础课

第 09 讲

实验设计：Activation Function

实验原则：先提出问题和预测，再手写 NumPy，再看曲线与结果，最后用 PyTorch 作为对照验证。

实验设计文档

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

1 实验目标

本实验不是“调用五个 API 看结果”，而是验证五个可以明确观察的现象：

- 1. 不同 Activation 对同一组输入产生不同数值映射；
- 2. Activation 通常逐元素工作，Shape 保持不变；
- 3. ReLU 是硬截断，GELU / SiLU 是平滑削弱；
- 4. 加入 ReLU 后，两层网络不能再被一个 affine 全局精确折叠；
- 5. 手写 NumPy 与 PyTorch 实现应数值一致。

2 实验问题与假设

编号	问题	预期/假设
E1	输入 $[-3, -1, 0, 1, 3]$ 后五种函数输出有何差异？	ReLU 抹去负数；Sigmoid/Tanh 压缩；GELU/SiLU 平滑保留部分负值。
E2	Activation 会改变 Shape 吗？	对常见逐元素 Activation，Shape 不变。
E3	ReLU/GELU/SiLU 在 0 附近有什么视觉差异？	ReLU 有硬折点；GELU/SiLU 更平滑。
E4	$2\text{ReLU}(3x - 1) + 0.5$ 能否被单一直线精确拟合？	不能；最大误差明显大于 0。
E5	NumPy 与 PyTorch 实现是否一致？	差异仅为浮点误差。

3 环境

最低要求：

- Python 3；
- NumPy；
- Matplotlib；
- PyTorch 可选，只用于最后验证。

运行：

```
python lesson09_activation_functions_experiment.py
```

4 实验一：手写五种 Activation

核心函数：

```
def relu(x):  
    return np.maximum(0.0, x)  
  
def sigmoid(x):  
    return 1.0 / (1.0 + np.exp(-x))  
  
def silu(x):  
    return x * sigmoid(x)
```

GELU 使用常见 $\tanh$ 近似。当前目标不是记复杂公式，而是知道：我们可以不用框架，直接按数学定义计算。

已运行结果

```
输入 x      = [-3. -1.  0.  1.  3.]  
ReLU(x)     = [0.  0.  0.  1.  3.]  
Sigmoid(x)  = [0.047426 0.268941 0.5 0.731059 0.952574]  
Tanh(x)     = [-0.995055 -0.761594 0. 0.761594 0.995055]  
GELU(x)     = [-0.003637 -0.158808 0. 0.841192 2.996363]  
SiLU(x)     = [-0.142278 -0.268941 0. 0.731059 2.857722]
```

5 实验二：曲线观察

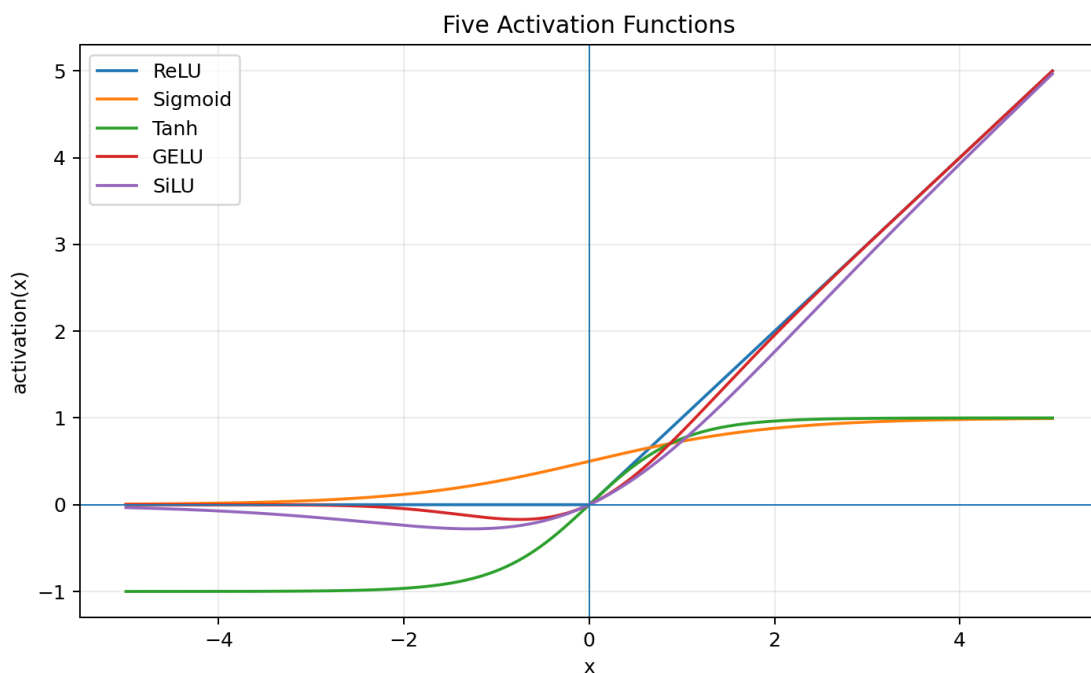


图 1: 五种 Activation 的整体曲线。

观察任务：

1. 找到 Sigmoid 的水平中心点；
2. 找到 Tanh 的原点；
3. 看 ReLU 在哪里出现折点；
4. 比较 GELU/SiLU 与 ReLU 的负数区域。

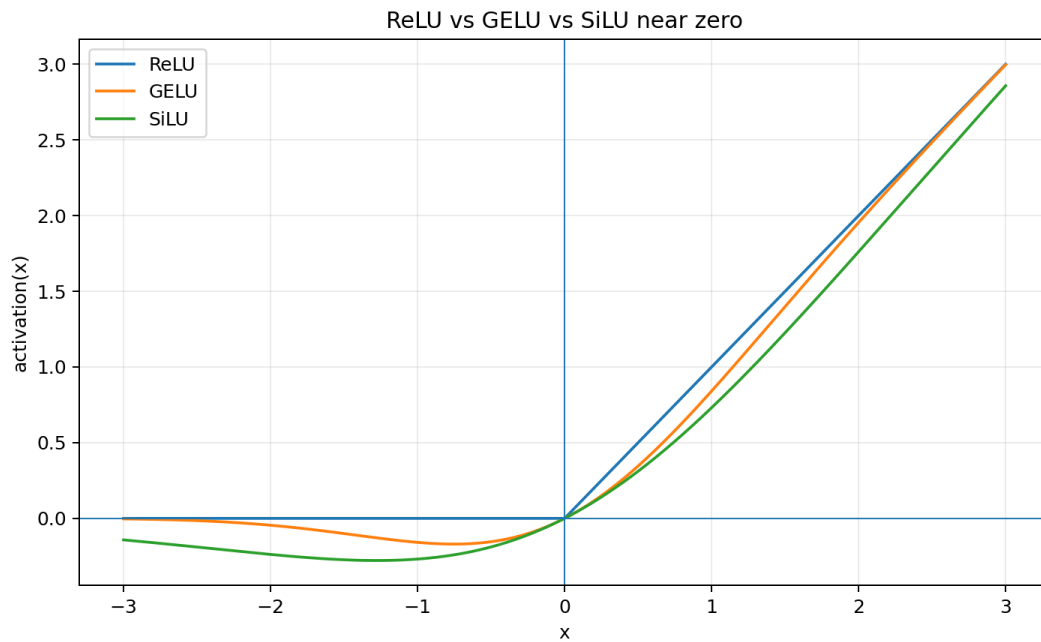


图 2: 0 附近的 ReLU / GELU / SiLU。

6 实验三：Linear -> Activation 的 Shape

输入：

$$X : (2, 3), \quad W : (3, 4), \quad b : (4,)$$

先算：

$$Z = XW + b : (2, 4)$$

再算 ReLU/GELU/SiLU。

实际输出：

```
X.shape      = (2, 3)
W.shape      = (3, 4)
b.shape      = (4,)
Z.shape      = (2, 4)
ReLU(Z).shape = (2, 4)
GELU(Z).shape = (2, 4)
SiLU(Z).shape = (2, 4)
```

核心概念

实验支持假设：逐元素 Activation 改变数字，但没有改变矩阵的行列组织。

7 实验四：非线性打破 affine 折叠

定义：

$$y = 2\text{ReLU}(3x - 1) + 0.5$$

程序使用最小二乘寻找最佳单一直线：

$$y = ax + c$$

结果：

最佳单一直线： $y = 2.264135 * x + 2.612475$

最大绝对误差：3.359255

若整个网络可完全折叠为单一 affine，最大误差应接近浮点误差 0；实际误差明显不为 0。

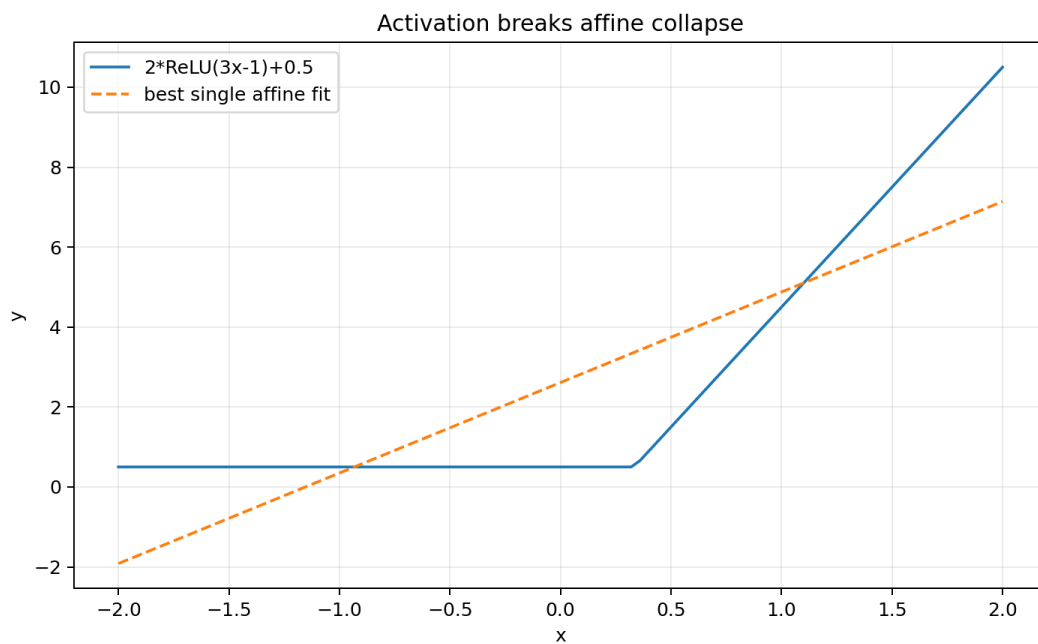


图 3: 非线性网络与最佳单一直线的差异。

8 实验五：PyTorch 对照

程序使用：

```
F.relu(z_t)
F.gelu(z_t, approximate="tanh")
F.silu(z_t)
```

实际最大差：

ReLU: 0.0
GELU: 8.33e-17
SiLU: 4.44e-16

这是正常浮点误差，可以认为结果一致。

9 如何写实验记录

建议记录四列：

实验	预测	实际结果	解释
E1	GELU 保留小负值	$-1 \mapsto -0.159$	平滑而非硬截断
E2	Shape 不变	$(2, 4) \rightarrow (2, 4)$	逐元素操作
E4	不能精确折叠	最大误差 3.359	加入了非线性折点

科研视角

科研实验的核心不是“代码跑通”，而是：事前问题明确、变量可控、结果可复现、结论只说数据支持的范围。即使是今天的小实验，也沿用这一结构。

10 扩展实验（可选）

只做一项即可：把输入范围从 $[-5, 5]$ 扩大到 $[-20, 20]$ ，观察 Sigmoid/Tanh 的“饱和”视觉效果。先写预测，再运行代码。

11 完成标准

你能独立回答：

- 为什么 Activation 不只是“多一个函数名”？
- 为什么 Shape 通常不变？
- 为什么 ReLU 能打破上一讲的 affine 折叠？
- 为什么 NumPy 手写与 PyTorch API 应得到一致数学结果？